



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ**

КУРСОВ ПРОЕКТ

Дисциплина: Програмни Среда

Тема:

**CVAnalysisHub – платформа за управление на видео анализи
с компютърно зрение, Blazor, Avalonia и две СУБД**

Изготвил:

Асен Пантелеев Попов

Фак. № 121223246

Група 38

III курс, КСИ

e-mail: asepopov@tu-sofia.bg

Преподавател:

гл. ас. П. Маринов

София, 2026 г.

Съдържание

1	Въведение	1
2	Използвани технологии	2
2.1	View технологии	2
2.1.1	Blazor	2
2.1.2	AvaloniaUI	2
2.2	Системи за управление на бази данни	3
2.2.1	SQLite	3
2.2.2	PostgreSQL	3
2.3	Други използвани технологии	3
2.3.1	Entity Framework Core	3
2.3.2	YoloDotNet, ONNX Runtime и ffmpeg	4
3	Какво беше необходимо, за да работят технологиите	6
3.1	Съгласуване на конфигурацията	6
3.2	Стартиране на настолния клиент и фонове услуги	7
3.3	Медийна последователност и обработка на видео	7
3.4	Работа под Linux и локална среда	7
4	Архитектура	8
4.1	Общ архитектурен модел	8
4.2	Слоеве на системата	10
4.2.1	CVAnalysisHub.Core	10
4.2.2	CVAnalysisHub.Application	10
4.2.3	CVAnalysisHub.Infrastructure	10
4.2.4	CVAnalysisHub.Web и CVAnalysisHub.Avalonia	10
4.2.5	Външно архитектурно резюме	11
4.3	Модел на базата данни	12
4.4	Последователност на анализа	12

5	Описание на реализацията	15
5.1	Разпределение на типовете по слоеве	15
5.2	Организация на преизползването на ViewModel-и	15
5.3	Организация на преизползването за двете СУБД	16
5.4	Организация на преизползваемите UI механизми	16
5.5	Ключови решения в кода	17
5.6	Трудности и преодоленни проблеми	26
6	Примери за използване	26
6.1	Пример за завършен анализ	26
6.2	Екрани за демонстрация	27
6.3	Демонстрация на преизползване	28
7	Заключение	30
	Литература	30

Глава 1 Въведение

CVAnalysisHub представлява курсов проект, насочен към управление на видео анализи с помощта на компютърно зрение. Практическата идея на системата е потребителят да може да качва локални видеа, да създава задачи за анализ върху тях, да следи изпълнението на тези задачи и да преглежда резултатите от разпознаването на обекти. Темата е избрана така, че да бъде едновременно достатъчно практическа и достатъчно богата архитектурно, за да позволи смислена реализация на пълноценен софтуерен сценарий.

Проектът не е замислен като обикновен CRUD пример. Неговата цел е да демонстрира как в рамките на .NET екосистемата може да се изгради приложение с реален домейн, ясно разделение на отговорностите и преизползване на логика между няколко технологични слоя. В случая това преизползване се проявява в две направления. От една страна едни и същи споделени ViewModel-и обслужват два различни потребителски интерфейса. От друга страна едни и същи абстракции на модела и бизнес логиката работят с две различни системи за управление на бази данни.

Функционално системата обединява няколко последователни сценария. Първо се качва изходно видео, което се съхранява във файловото хранилище на приложението. След това върху него се създава задача за анализ. Тази задача се обработва асинхронно от фонов обработчик, който извиква избрания модул за инференция. При реалния сценарий с YoloDotNet се извличат подобрени кадри, върху тях се изпълнява разпознаване на обекти, резултатите се записват в базата данни и накрая се генерира аотирано изходно видео. Потребителят може да преглежда както историята от анализи, така и детайлите за конкретно изпълнение, включително откритите обекти и получения видео резултат.

Архитектурно проектът е реализиран така, че логиката на представяне да бъде максимално споделена, а специфичните за конкретната технология елементи да останат тънък слой върху общия приложен слой. Именно поради това структурата на проекта не е сведена до механични папки от тип Views, ViewModels и Models, а е организирана в по-реалистични слоеве: Core, Application, Infrastructure, Web и Avalonia. Тази организация позволява по-ясно разделение между домейн модела, приложната логика, инфраструктурните зависимости и визуализацията.

Настоящата документация има за цел да представи както крайния резултат, така и инженерните решения зад него. По-конкретно се разглеждат избраните технологии, условията за използването им, архитектурната организация на проекта, начинът на преизползване на ViewModel-и и на СУБД, преизползваемите механизми за филтриране и визуализация на списъци, както и примерни сценарии на работа. В края са обобщени и основните изводи от разработката, включително трудностите, които се

наложи да бъдат преодолены.

Исходният код, README описанието и документацията на проекта са публикувани в GitHub репото <https://github.com/Apsie09/CV-Analysis-Hub>.

Глава 2 Използвани технологии

2.1 View технологии

2.1.1 Blazor

Blazor е избран като уеб технология в проекта, тъй като позволява изграждане на интерактивен потребителски интерфейс изцяло в .NET среда, без необходимост от отделен приложен слой на JavaScript. За конкретната разработка това се оказва особено удобно, защото споделените ViewModel-и могат да бъдат инжектирани директно в компонентите и използвани като основен източник на състоянието на интерфейса. Освен това Blazor е много подходящ за изграждане на преизползваеми елементи на интерфейса, каквито в проекта са панелът за филтриране и компонентът за визуализация на списъци.

Blazor е полезен не само като технология за потребителски интерфейс, а и като добра среда за изграждане на автоматизирано генериран потребителски интерфейс върху общи приложни абстракции. Компонентният модел на Blazor улеснява изграждането на повторно използвани визуални елементи, които работят върху метаданни и ViewModel-и, а не върху конкретни класове от модела. Това го прави подходящ избор за филтри, таблици и детайлни изгледи, които се повтарят в различни части на приложението.

2.1.2 AvaloniaUI

Avalonia е избрана като настолна технология, защото предоставя XAML, модел на обвързване и команди, близък до класически MVVM инструменти като WPF, но без да бъде ограничена само до Windows. Това е важен практически аргумент, тъй като проектът се разработва и тества под Linux. В тази среда Avalonia позволява реална настолна реализация със сравнително тънък code-behind и естествено обвързване със споделен ViewModel слой.

В контекста на настоящия проект Avalonia е използвана като пълноценен настолен клиент, а не само като минимална визуална обвивка. Настолният клиент реално поддържа обзорни метрики, преглед на видео библиотеката, качване на видеа, стартиране на анализи, разглеждане на историята от изпълнения и визуализация на

резултатите. По този начин се демонстрира, че споделените приложни ViewModel-и могат да обслужват втори интерфейс с реална функционалност.

2.2 Системи за управление на бази данни

2.2.1 SQLite

SQLite е подходяща за локална разработка и за демонстрационни сценарии, тъй като не изисква отделен сървър и работи чрез един файл. Това я прави удобна за бързо стартиране на приложението, за преносима демонстрационна конфигурация и за ситуации, в които е важно системата да може да бъде показана без допълнителна инсталация на сървър за база данни. В рамките на проекта SQLite е естествен избор за лек режим на работа и за лесна повтораемост при локално тестване.

Същевременно SQLite има и ограничения. Тя не отразява напълно типична експлоатационна среда и не е най-подходящият избор за по-сериозен многопотребителски сценарий. Именно поради това в проекта е включена и PostgreSQL, така че слоят за съхранение да не бъде обвързан само с една удобна за разработка технология.

2.2.2 PostgreSQL

PostgreSQL е избрана като втора СУБД, защото е значително по-близка до реална сървърна среда. Тя предоставя по-богата екосистема, по-реалистичен модел на внедряване и добра интеграция с EF Core чрез Npgsql. В контекста на проекта PostgreSQL показва, че една и съща приложна и бизнес логика може да работи върху различни доставчици без дублиране на код.

Най-същественото тук е, че проектът не поддържа две бази чрез два отделни service слоя. Вместо това същият `AppDbContext`, същите инфраструктурни услуги и същите приложни ViewModel-и работят и с SQLite, и с PostgreSQL. Разликата се свежда до конфигурационен избор на доставчик, а не до паралелни имплементации със същата логика.

2.3 Други използвани технологии

2.3.1 Entity Framework Core

Entity Framework Core е основата на слоя за съхранение. Чрез него проектът получава общ модел на данните, общ механизъм за заявки и възможност за превключване между SQLite и PostgreSQL. За конкретната разработка EF Core е ценен и с това, че позволява приложната логика да остане сравнително чиста и независима от

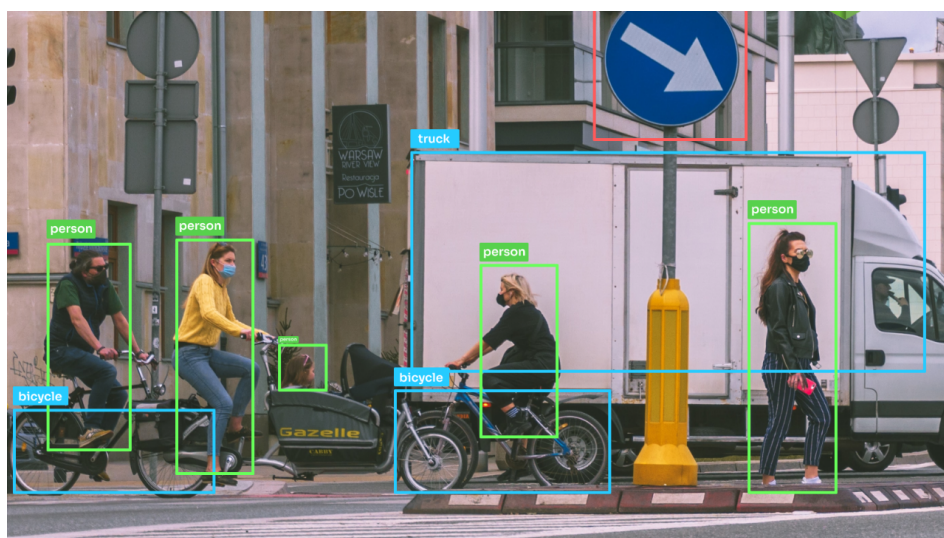
конкретния доставчик на база данни, докато инфраструктурният слой поема конфигурацията и реалната интеграция с базата.

2.3.2 YoloDotNet, ONNX Runtime и ffmpeg

Важна особеност на проекта е, че частта за видео анализ не е оставена на ниво симулация. Вместо фиктивни записи за разпознати обекти е реализирана работеща последователност с YoloDotNet, ONNX модел и ffmpeg. ffmpeg се използва за извличане на подбрани кадри и за рендериране на аотирано изходно видео, а YoloDotNet изпълнява разпознаване на обекти върху тези кадри. Именно тази комбинация превръща приложението от обикновен административен интерфейс в система с реална аналитична функционалност.

Официалният проект на YoloDotNet го представя като модулна .NET библиотека за YOLO инференция, която стъпва върху ONNX Runtime и е насочена към предвидимо и производително изпълнение без Python среда за изпълнение и без тежки външни библиотеки за компютърно зрение [12]. Тази идея съответства много добре на целите на настоящия проект, защото CVAnalysisHub също се стреми към изцяло .NET реализация, която да бъде лесна за демонстрация и достатъчно компактна за учебна среда.

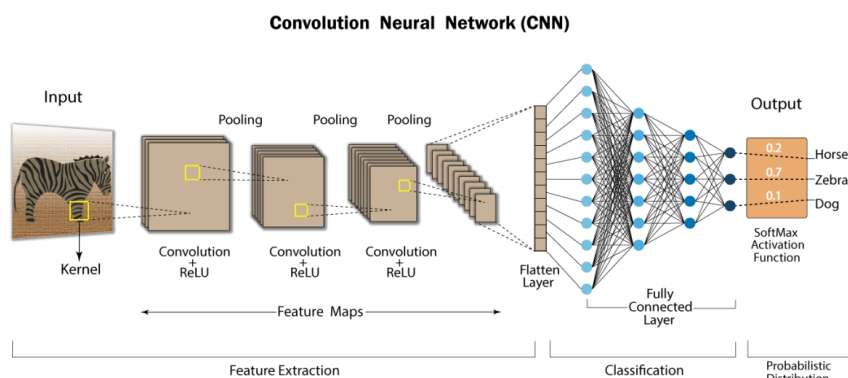
Разпознаването на обекти представлява задача от компютърното зрение, при която системата трябва едновременно да определи какъв обект присъства в изображението и къде точно се намира той. За разлика от обикновената класификация, тук не е достатъчно да се каже, че в кадъра има човек или автомобил. Необходимо е и локализиране на всеки обект чрез ограничителна рамка, както и оценка на увереността на модела. Именно този тип резултат се съхранява в CVAnalysisHub под формата на етикет, увереност и координати на ограничителната рамка.



Фигура 2.1: Пример за резултат от разпознаване на обекти с bounding boxes

В основата на много modern computer vision модели стоят конволюционните невронни

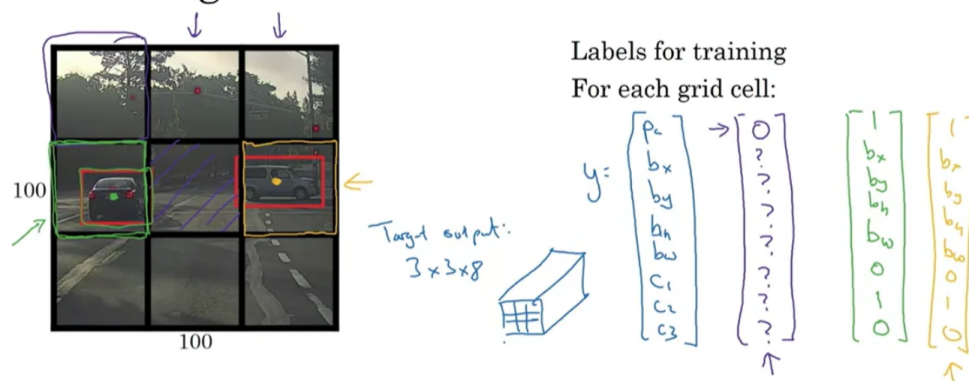
мрежи. Тяхната идея е, че чрез конволюционни филтри могат да се извличат локални визуални признаци като контури, текстури и по-сложни пространствени структури. С нарастване на дълбочината на мрежата тези признаци стават все по-абстрактни и подходящи за разпознаване на цели обекти. Макар съвременните YOLO архитектури да съдържат и допълнителни оптимизации, връзката им с CNN подхода остава фундаментална, защото те също използват learnable feature extraction върху изображението, за да достигнат до финално предсказване.



Фигура 2.2: Схематично представяне на конволюционна невронна мрежа

YOLO, съкратено от *You Only Look Once*, е семейство от едностъпкови модели за разпознаване на обекти. Основната идея е, че вместо да се правят отделни етапи за генериране на предложения за региони и последваща класификация, моделът обработва изображението в един общ проход за инференция и директно предсказва класове и ограничителни рамки. Това го прави особено подходящ за приложения, в които е важен балансът между точност и скорост, например обработка на видео и почти реално време.

YOLO algorithm



[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Andrew Ng

Фигура 2.3: Схема на общата идея зад YOLO алгоритмите

В проекта е избрана серията YOLOv8. Причината не е единствено популярността

на модела, а добрият практически баланс между зрялост на екосистемата, стабилна документация, лесен export към ONNX и широка поддръжка в инструменти като Ultralytics и YoloDotNet. Официалната екосистема на Ultralytics представя YOLOv8 като state-of-the-art поколение, създадено за задачи като detect, segment, classify и pose, с няколко размерни варианта, които позволяват избор според конкретния хардуер и натоварване [13, 14].

По-конкретно в CVAnalysisHub се използва `yolov8n.onnx`, тоест nano вариантът на YOLOv8. Този избор е напълно логичен за учебен проект, който трябва да работи надеждно на стандартна машина и в настолно приложение с CPU доставчик за изпълнение. Според публикуваните от Ultralytics сравнения nano моделът е най-малкият вариант по брой параметри и изчислителна сложност, като същевременно остава практически полезен за реални задачи по детекция [14]. С други думи, избраният модел не максимизира абсолютната точност, а търси добър инженерен компромис между време за инференция, използвана памет и достатъчно добри резултати за демонстрация върху видео.

Този избор на технологии носи и практическа стойност за документацията. Проектът показва не само UI и работа с база данни, а и интеграция с външен инструмент и с библиотека за машинно зрение. Следователно той обхваща по-широк спектър от инженерни задачи, отколкото стандартен учебен пример.

Глава 3 Какво беше необходимо, за да работят технологиите

Освен видимата функционалност, проектът включва и интеграционна работа, без която крайният резултат не би бил възможен. В настоящия проект именно тази невидима на пръв поглед работа се оказва съществена. Архитектурно кодът можеше да изглежда правилен още в ранна фаза, но реалното съгласуване между уеб и настолния клиент, между двете бази данни и между обработката на видео и потребителския интерфейс изискваше допълнителни решения.

3.1 Съгласуване на конфигурацията

За да могат двата клиента да показват еднакви данни и да работят последователно, се оказва необходимо и двата да сочат към една и съща конфигурационна среда. Това включва не само избора на `PersistenceProvider`, но и еднакъв низ за свързване, еднакво файлово хранилище, еднакъв път към ONNX модела и еднаква конфигурация за `ffmpeg`. Без това приложението би изглеждало функционално, но двата клиента биха могли да показват различни данни и така да създадат погрешно впечатление

за архитектурно разминаване.

Двете UI технологии не са реализирани като две независими програми, а като две визуализации върху споделена приложна логика. Аналогично, двете СУБД не са представени като два отделни проекта, а като един модел и един бизнес слой, който работи и с двата доставчика. Следователно правилната конфигурация е част от архитектурната цялост на системата, а не просто техническа подробност.

3.2 Стартиране на настолния клиент и фонові услуги

При Avalonia се появи специфичен проблем. Ако настолното приложение използва само ръчно конструиран `ServiceCollection`, фонові услуга като обработчик на анализи няма да бъде стартирана по същия начин, както в уеб приложението. За да се избегне това разминаване, Avalonia клиентът беше свързан към `IHost`. По този начин настолното приложение започна да използва същия модел за инжектиране на зависимости, фонові услуги и конфигурация, който вече съществуваше в уеб слоя.

Това решение се оказа ключово, защото позволи `QueuedAnalysisBackgroundService` да работи и в настолната среда и доближи стартирането на двата клиента. Така Avalonia приложението се превърна в реално функционален интерфейс със собствена роля в системата.

3.3 Медийна последователност и обработка на видео

Друг важен слой от работа беше свързан с реалната обработка на видео. За да не остане приложението на ниво административен интерфейс с фиктивни резултати, беше необходимо качените файлове действително да се записват във файлово хранилище, от тях да се извличат кадри чрез `ffmpeg`, тези кадри да се подават към YOLO модела и накрая да се генерира аотирано изходно видео.

Тази интеграция не е видима директно от структурата на решението, но е сърцевината на функционалността. Тя обвързва файловата система, външния инструмент за обработка на видео, модула за inferенция и слоя за съхранение в един последователен процес. Без такава реализация приложението би имало добра архитектура, но не би показвало реален аналитичен резултат.

3.4 Работа под Linux и локална среда

Фактът, че разработката и тестването се извършват под Linux, постави и няколко практически условия. Наложиха се да бъде конфигуриран коректен път към

/usr/bin/ffmpeg, да се вземе предвид поведението на локалните HTTPS сертификати в браузъра и да се намери подходящ настолен подход за визуализация на видео без въвеждане на тежка зависимост за вградено възпроизвеждане. В резултат настолният клиент комбинира предварителни кадри и отваряне на оригиналното или обработеното видео чрез системния видео плейър.

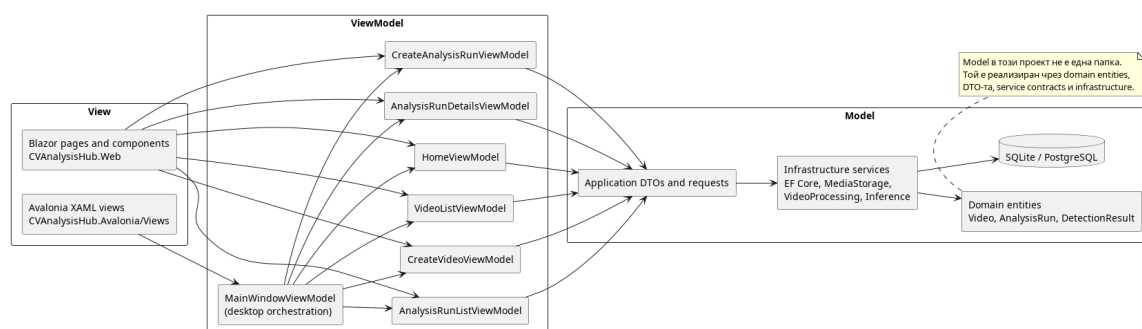
Тук ясно се вижда разликата между това кодът да е написан и това приложението да е действително работещо в реална среда. Именно тези интеграционни аспекти често не се забелязват в крайния продукт, но представляват съществена част от извършената инженерна работа.

Глава 4 Архитектура

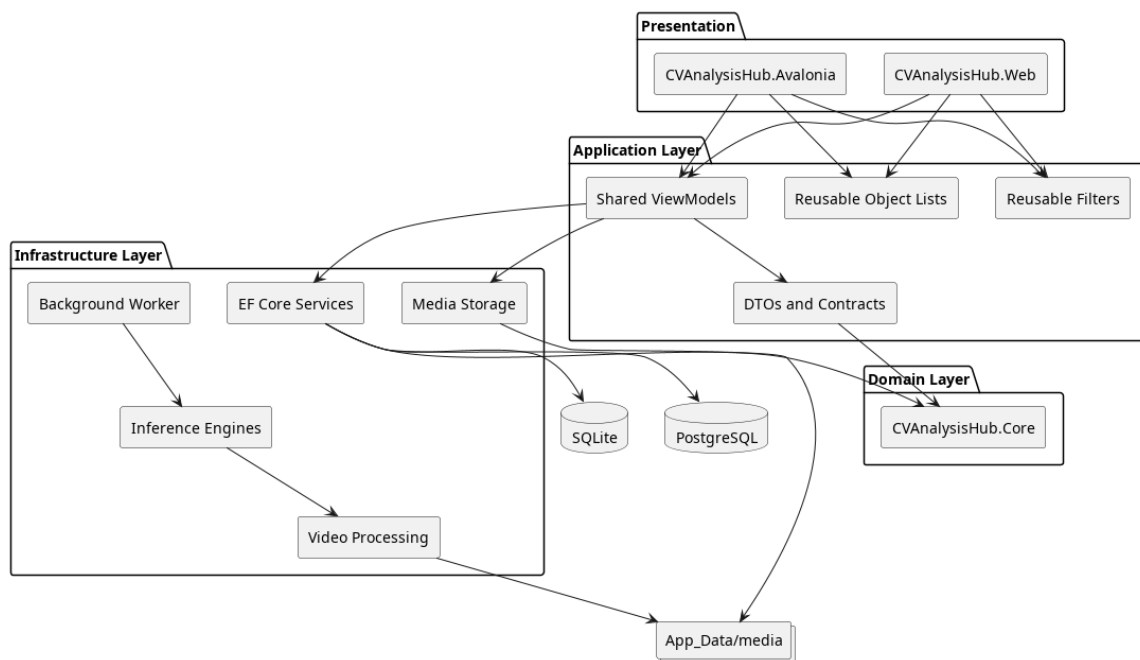
4.1 Общ архитектурен модел

Проектът използва слоева архитектура с MVVM на ниво представяне. Това означава, че ролите на *View*, *ViewModel* и *Model* са налице, но не са материализирани като три механични папки в един и същ проект. Вместо това те са разпределени по слоеве според отговорностите си. View частта се реализира от Blazor компонентите и Avalonia XAML екраните. ViewModel частта е концентрирана основно в приложния слой чрез споделени ViewModel-и, а при настолния клиент има допълнителен координиращ ViewModel за специфично поведение на интерфейса. Model частта е по-широко понятие и включва домейн обекти, DTO-та, заявки, service contract-и и инфраструктурни услуги.

Този подход е по-подходящ за реален .NET проект, отколкото просто физическо групиране по три папки, защото позволява домейн логиката да остане изолирана от грижите на потребителския интерфейс, а споделената приложна логика да бъде преизползвана от повече от една View технология. Диаграмата на [фигура 4.1](#) показва точно това съответствие между MVVM ролите и конкретните части на проекта, докато [фигура 4.2](#) представя по-общия слоев изглед на системата.

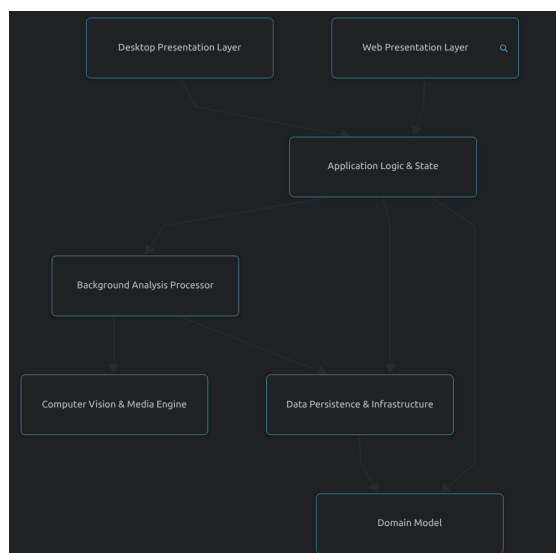


Фигура 4.1: MVVM картографиране в проекта: кои части са View, ViewModel и Model



Фигура 4.2: Обща архитектурна диаграма на проекта

Към собствените диаграми в документацията може да се добави и външен архитектурен поглед върху репото, генериран чрез [CodeBoarding](#). Тази визуализация не замества ръчния архитектурен анализ, а служи като бърза карта за ориентация в структурата на проекта и в зависимостите между основните му слоеве.



Фигура 4.3: Външно архитектурно обобщение на CVAnalysisHub, получено чрез CodeBoarding

4.2 Слоеве на системата

4.2.1 CVAnalysisHub.Core

CVAnalysisHub.Core е домейн слой на проекта. В него се намират основните бизнес обекти Video, AnalysisRun и DetectionResult. Тези типове описват бизнес данните и връзките между тях, без да зависят нито от конкретна технология за потребителски интерфейс, нито от конкретен доставчик на база данни. Така те изпълняват ролята на стабилно ядро, върху което се изграждат останалите слоеве.

4.2.2 CVAnalysisHub.Application

CVAnalysisHub.Application съдържа приложни абстракции и споделена логика на представяне. В този слой се намират DTO-тата, моделите на заявки, service contract-ите, споделените ViewModel-и, както и произползваемите абстракции за филтри и за списъци от обекти. Именно тук е реализирана същинската идея за произползване на ViewModel-и от различни View технологии. Вместо всяка технология да получи собствен набор от класове със сходна функционалност, и уеб, и Avalonia клиентът използват един и същ приложен слой.

4.2.3 CVAnalysisHub.Infrastructure

Infrastructure слойът съдържа конкретните имплементации на услугите за съхранение, медийна обработка и анализ. Тук се намират EF Core контекстът, service класовете, конфигурацията на доставчика на база данни, файловото хранилище, логиката за обработка на видео, модулите за инференция и фоновият обработчик за чакащите анализи. Това е слой, който осигурява взаимодействието с външните технологии, без да пренася техните зависимости към домейн и приложния слой.

4.2.4 CVAnalysisHub.Web и CVAnalysisHub.Avalonia

CVAnalysisHub.Web представлява уеб слой на представяне и съдържа Blazor интерфейса, произползваеми Razor компоненти и composition root-а за уеб изпълнение. CVAnalysisHub.Avalonia е настолният слой на представяне и съдържа Avalonia XAML View-та, както и минимален специфичен за технологията слой за координация на настолния интерфейс. Съществено тук е, че тези два проекта изпълняват различни View роли, но не дублират основната логика на представяне, а я използват от общия приложен слой.

4.2.5 Външно архитектурно резюме

CodeBoarding дава на разработчици и coding agents визуална карта на дадена кодова база. Инструментът комбинира статичен анализ с LLM reasoning, за да генерира архитектурни диаграми, документация на ниво компоненти и навигируеми изходи, които могат да се използват в IDE, CI процеси и документация [2]. В [фигура 4.3](#) той е използван като допълнителен поглед към структурата на CVAnalysisHub, а не като заместител на ръчния архитектурен анализ. CodeBoarding е стартър, започнат от Иван Милев, който е възпитаник на Технически университет – София, продължава обучението си с магистратура в ETH Zurich и към момента развива проекта в Силициевата долина.

Application Logic & State CodeBoarding обобщава този слой като мястото, в което се оркестрира бизнес логиката и се управлява състоянието на интерфейса чрез shared ViewModel-и и service абстракции. Това съответства пряко на ролята на CVAnalysisHub.Application.

Background Analysis Processor Тук инструментът разпознава жизнения цикъл на задачите за анализ и координацията между медийния двигател и persistence слоя чрез фонов обработчик. Това отговаря на hosted service слоя за обработка на чакащи анализи и на свързаните processing услуги.

Web Presentation Layer Уеб клиентът е разпознат като Blazor слой на представяне, който управлява потребителските взаимодействия и браузърното изпълнение. Това е CVAnalysisHub.Web.

Desktop Presentation Layer Настолният клиент е разпознат като Avalonia слой на представяне, който споделя приложната логика с уеб клиента чрез общите ViewModel-и. Това е CVAnalysisHub.Avalonia.

Computer Vision & Media Engine CodeBoarding отделя тежката медийна обработка, извличането на кадри и ML детекцията чрез YOLO и ONNX runtime. Това съвпада с VideoProcessingService и YoloDotNetAnalysisInferenceEngine.

Data Persistence & Infrastructure Този слой е описан като отговорен за съхранението и извличането на данни чрез Entity Framework Core и за реализацията на service contract-ите към SQLite или PostgreSQL. Това е ролята на CVAnalysisHub.Infrastructure.

Domain Model Домейн моделът е разпознат като ядро от бизнес обекти, описващи основните понятия в системата, като Video, AnalysisRun и DetectionResult. Това съответства на CVAnalysisHub.Core.

4.3 Модел на базата данни

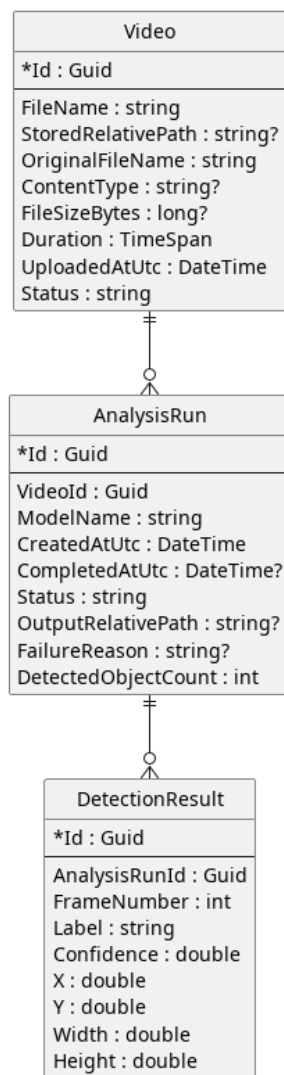
Моделът на съхранение е умишлено компактен. Той не е прекалено сложен, защото проектът няма нужда от голям брой таблици, а от ясна структура, която поддържа основния бизнес сценарий. Моделът е достатъчно богат, за да реализира реален работен процес за видео анализ, и в същото време остава достатъчно малък, за да бъде лесен за обяснение и демонстрация.

Схемата съдържа три основни таблици. Таблицата **Video** описва качените видеа и пази връзка към файловото хранилище. Таблицата **AnalysisRun** представя конкретно изпълнение на анализ върху дадено видео и пази информация за модела, времето на изпълнение, статуса, броя открити обекти и изходния видео резултат. Таблицата **DetectionResult** пази единичните детекции за конкретно изпълнение, включително номер на кадър, етикет, увереност и координати на ограничителната рамка. По този начин едно видео може да има множество изпълнения на анализ, а всяко изпълнение може да съдържа множество резултати от детекция.

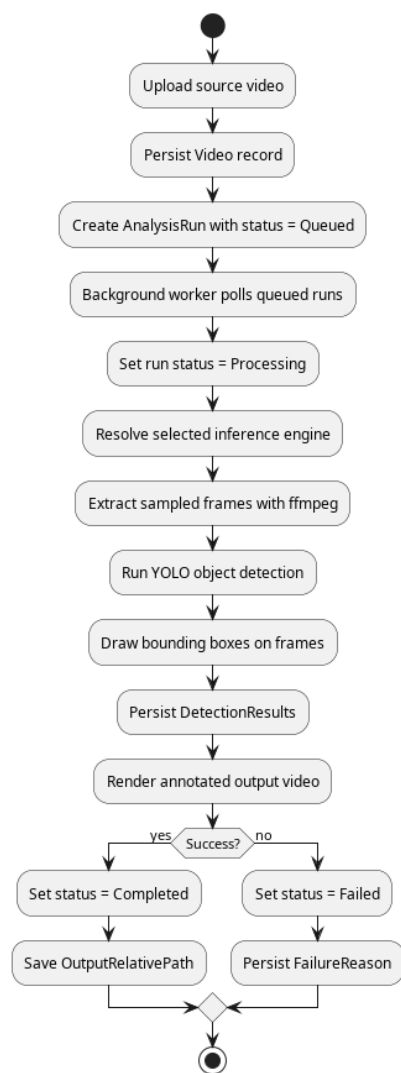
4.4 Последователност на анализа

Една от важните архитектурни характеристики на проекта е, че последователността на анализа не е реализирана като директно синхронно действие от слоя на потребителския интерфейс. Вместо това процесът е разделен на ясни етапи. Първо потребителят качва видео и създава задача за анализ. След това се създава **AnalysisRun** със статус **Queued**. Фоновият обработчик периодически проверява за чакащи задачи, избира следващата и я премества в състояние **Processing**. Източникът се подава към избрания модул за инференция, който извлича кадри, изпълнява детекция, създава резултатите и накрая генерира аотирано изходно видео. В зависимост от изхода статусът се променя на **Completed** или **Failed**.

Този модел прави системата по-близка до реален работен процес, отколкото до демонстрационен сценарий само за настолен клиент. Освен това той отделя потребителския интерфейс от тежката обработка и позволява по-чиста организация на отговорностите между слоя на представяне, приложния слой и инфраструктурния слой.



Фигура 4.4: ER диаграма на основните таблици в проекта



Фигура 4.5: Диаграма на анализа на видео в проекта

Глава 5 Описание на реализацията

5.1 Разпределение на типовете по слоеве

За да бъде архитектурата ясна и лесна за поддръжка, типовете в проекта са разпределени по слоеве според тяхната функция. В домейн слоя се намират основните обектни типове, които описват същинските бизнес обекти: видео, анализ и резултат от детекция. Тези класове не познават конкретна технология за потребителски интерфейс, нито начин на съхранение, което позволява да служат като стабилен модел на системата.

Приложният слой съдържа класовете, които оформят интерфейса между домейна и технологията за представяне. Тук се намират DTO-тата, моделите на заявки, service contract-ите и споделените ViewModel-и като `HomeViewModel`, `VideoListViewModel`, `CreateVideoViewModel`, `AnalysisRunListViewModel`, `CreateAnalysisRunViewModel` и `AnalysisRunDetailsViewModel`. В същия слой са разположени и преизползваемите абстракции за филтри и за списъци от обекти, които позволяват една и съща логика да се прилага върху различни екрани.

Infrastructure слойът съдържа реалните имплементации, върху които стъпва приложението: `AppDbContext`, EF Core service класовете, услугите за съхранение на файлове, логиката за обработка на видео, фоновата услуга и модулите за инференция. Така проектът остава ясно структуриран: приложният слой описва какво трябва да може системата, а инфраструктурният слой реализира как точно става това.

5.2 Организация на преизползването на ViewModel-и

Изискването за едни и същи ViewModel-и в две различни View технологии е едно от най-съществените за проекта. Вместо две паралелни дървета от класове със сходна функционалност, едното за уеб, а другото за настолен интерфейс, споделените ViewModel-и се намират в `CVAnalysisHub.Application` и се използват и от двата клиента.

Това преизползване е приложено в основните функционални сценарии на приложението. Обзорните метрики, списъците от видеа и анализи, филтрите, процесът на качване, процесът по създаване на анализ и детайлният преглед на избран analysis run са реализирани чрез споделени приложни ViewModel-и. Avalonia има собствен `MainWindowViewModel`, но неговата роля не е да дублира бизнес функционалността, а да координира специфичното за настолния клиент състояние на интерфейса върху

тези споделени ViewModel-и, например предварителни кадри и действия за отваряне на изходното и обработеното видео.

5.3 Организация на преизползването за двете СУБД

Поддръжката на две СУБД е реализирана така, че да не се нарушава идеята за общ модел и обща бизнес логика. Не съществува отделен service слой за SQLite и отделен service слой за PostgreSQL. Вместо това изборът на доставчик се извършва конфигурационно в инфраструктурния слой. Там, въз основа на стойността на `Persistence:Provider`, се избира дали EF Core да използва `UseSqlite` или `UseNpgsql`.

Тази организация избягва дублиране на една и съща логика за всяка СУБД. В текущия проект домейн и приложният слой не се интересуват кой точно доставчик стои отдолу. За тях услугата за съхранение е една и съща по поведение, което позволява системата да бъде стартирана и с SQLite, и с PostgreSQL без промяна във ViewModel-ите и без пренаписване на бизнес логиката.

5.4 Организация на преизползваемите UI механизми

Преизползваемата функционалност за филтриране е реализирана чрез общ модел на метаданни за полетата на филтрите. Всеки екран с търсене описва полетата си чрез надпис, описание, тип на входа и подсказващи текстове. Въз основа на тези метаданни преизползваемият визуализатор избира подходяща контрола, например текстово поле, падащ списък, числов интервал или интервал от дати. Въведените стойности не остават на ниво интерфейс, а се трансформират в типизирана заявка за търсене, която се използва от слоя за съхранение за изграждане на заявка към базата данни. Най-важното е, че тази функционалност не е написана за една конкретна таблица, а е демонстрирана върху два различни модела: `Videos` и `AnalysisRuns`.

По сходен начин е реализирана и преизползваемата функционалност за визуализиране на списъци от обекти. На преизползваемия `list ViewModel` се подава списък от DTO обекти, конфигурират се видимите колони и при избор на ред може да се върне целият избран обект, независимо кои свойства са били визуализирани. Визуализацията може да се изчиства и зарежда отново с нов списък. Същият механизъм е приложен върху `Videos` и `AnalysisRuns`, което показва, че таблицата и `selection` логиката не са обвързани с един конкретен модел.

5.5 Ключови решения в кода

Едно от най-важните архитектурни решения е превключването между SQLite и PostgreSQL. В проекта това не е реализирано чрез два различни слоя за съхранение, а чрез централизирана конфигурация в инфраструктурния слой. Така приложната логика остава една и съща, а конкретният доставчик се избира в стартовата конфигурация.

```
1      public static IServiceCollection AddInfrastructure(  
2          this IServiceCollection services,  
3          IConfiguration configuration)  
4      {  
5          var persistenceOptions = configuration  
6              .GetSection(PersistenceOptions.SectionName)  
7              .Get<PersistenceOptions>() ?? new PersistenceOptions();  
8          var provider = persistenceOptions.Provider.Trim();  
9          var connectionStringName = ResolveConnectionStringName(provider);  
10         var connectionString = configuration.GetConnectionString(  
11             connectionStringName);  
12  
13         if (string.IsNullOrEmpty(connectionString))  
14         {  
15             throw new InvalidOperationException(  
16                 $"Connection string '{connectionStringName}' is not  
17                 configured for provider '{provider}'");  
18         }  
19  
20         services.Configure<ComputerVisionOptions>(configuration.GetSection(  
21             "ComputerVision"));  
22         services.Configure<MediaStorageOptions>(configuration.GetSection(  
23             "MediaStorage"));  
24         services.Configure<VideoProcessingOptions>(configuration.GetSection(  
25             "VideoProcessing"));  
26         services.Configure<PersistenceOptions>(configuration.GetSection(  
27             PersistenceOptions.SectionName));  
28         services.AddDbContextFactory<AppDbContext>(options =>  
29             ConfigureDbContext(options, provider, connectionString));  
30         services.AddScoped<AppDbInitializer>();  
31         services.AddSingleton<MediaStorageService>();  
32         services.AddSingleton(sp => sp.GetRequiredService<Microsoft.  
33             Extensions.Options.IOptions<VideoProcessingOptions>>().Value);  
34         services.AddSingleton<VideoProcessingService>();  
35         services.AddScoped<IAnalysisRunProcessor, EfCoreAnalysisRunProcessor  
36             >();  
37         services.AddScoped<IAnalysisRunService, EfCoreAnalysisRunService>();
```

```

29     services.AddSingleton<PlaceholderAnalysisInferenceEngine>();
30     services.AddSingleton<YoloDotNetAnalysisInferenceEngine>();
31     services.AddSingleton<IAnalysisInferenceEngine,
ConfigurableAnalysisInferenceEngine>();
32     services.AddScoped<IVideoService, EfCoreVideoService>();
33     services.AddHostedService<QueuedAnalysisBackgroundService>();
34
35     return services;
36 }
37
38 private static void ConfigureDbContext(
39     DbContextOptionsBuilder options,
40     string provider,
41     string connectionString)
42 {
43     switch (provider.Trim().ToLowerInvariant())
44     {
45         case "postgres":
46         case "postgresql":
47         case "npgsql":
48             options.UseNpgsql(connectionString);
49             break;
50         case "sqlite":
51             options.UseSqlite(connectionString);
52             break;
53         default:
54             throw new InvalidOperationException(
55                 $"Unsupported database provider '{provider}'. Supported
values are 'Sqlite' and 'PostgreSql'.");
56     }
57 }
58
59 private static string ResolveConnectionStringName(string provider)
60 {
61     return provider.Trim().ToLowerInvariant() switch
62     {
63         "postgres" or "postgresql" or "npgsql" => "PostgreSql",
64         "sqlite" => "Sqlite",
65         _ => throw new InvalidOperationException(
66             $"Unsupported database provider '{provider}'. Supported
values are 'Sqlite' and 'PostgreSql'.")
67     };
68 }

```

Код 5.1: Откъс от DependencyInjection.cs, който избира SQLite или PostgreSQL

Кодът от [listing 5.1](#) показва две важни идеи. Първо, низът за свързване се определя спрямо стойността на `Persistence:Provider`, а не е фиксиран в приложния слой. Второ, изборът между `UseSqlite` и `UseNpgsql` е концентриран на едно място. Именно това прави възможно изпълнението на едно и също приложение върху две СУБД без дублиране на бизнес логика.

Втората съществена част е слойът със споделени `ViewModel`-и. Следният откъс показва, че `VideoListViewModel` не е специфичен нито за Blazor, нито за Avalonia. Той комбинира споделени филтри, преизползваем списък от обекти и зареждане на данни през `service contract`, което му позволява да бъде използван от двата интерфейса.

```
1 public sealed class VideoListViewModel(IVideoService videoService) :
    ViewModelBase
2 {
3     private static readonly ObjectListColumnDefinition<VideoDto>[]
    ColumnDefinitions =
4     [
5         new("originalFileName", "Original file", video => video.
    OriginalFileName),
6         new("storedFileName", "Stored file", video => video.FileName),
7         new("duration", "Duration", video => video.Duration.ToString(@"hh\:
    mm\:ss")),
8         new("uploadedAtUtc", "Uploaded (UTC)", video => video.UploadedAtUtc.
    ToString("yyyy-MM-dd HH:mm")),
9         new("status", "Status", video => video.Status)
10    ];
11
12    private IReadOnlyList<VideoDto> videos = Array.Empty<VideoDto>();
13    private bool isLoading;
14
15    public VideoFilterViewModel Filters { get; } = new();
16
17    public ObjectListViewModel<VideoDto> Browser { get; } = new(
18        ColumnDefinitions,
19        ["originalFileName", "duration", "uploadedAtUtc", "status"]);
20
21    public IReadOnlyList<VideoDto> Videos
22    {
23        get => videos;
24        private set => SetProperty(ref videos, value);
25    }
26
27    public bool IsLoading
28    {
29        get => isLoading;
30        private set => SetProperty(ref isLoading, value);
```

```

31     }
32
33     public bool HasActiveFilters => Filters.HasActiveValues;
34
35     public VideoDto? SelectedVideo => Browser.SelectedItem;
36
37     public async Task LoadAsync(CancellationTokentoken cancellationTokentoken =
38     default)
39     {
40         IsLoading = true;
41
42         try
43         {
44             Videos = await videoService.SearchAsync(Filters.BuildRequest(),
45             cancellationTokentoken);
46             Browser.SetItems(Videos);
47         }
48         finally
49         {
50             IsLoading = false;
51         }
52     }
53
54     public void ClearDisplay()
55     {
56         Videos = Array.Empty<VideoDto>();
57         Browser.Clear();
58         OnPropertyChanged(nameof(SelectedVideo));
59     }
60
61     public Task ApplyFiltersAsync(CancellationTokentoken cancellationTokentoken =
62     default)
63     {
64         return LoadAsync(cancellationTokentoken);
65     }
66
67     public Task ClearFiltersAsync(CancellationTokentoken cancellationTokentoken =
68     default)
69     {
70         Filters.Clear();
71         return LoadAsync(cancellationTokentoken);
72     }
73
74     public void SetColumnVisibility(string columnKey, bool isVisible)
75     {

```

```

72     Browser.SetColumnVisibility(columnKey, isVisible);
73 }
74
75 public void SelectRow(ObjectListRowViewModel<VideoDto>? row)
76 {
77     Browser.SelectRow(row);
78     OnPropertyChanged(nameof(SelectedVideo));
79 }

```

Код 5.2: Откъс от VideoListViewModel.cs със shared list и filter логика

Откъсът от [listing 5.2](#) показва как използваемата функционалност е капсулирана на ниво ViewModel, а не само на ниво визуален компонент. Списъкът от видеа, активните филтри, зареждането на резултатите и изборът на ред са организирани в една споделена приложна абстракция, която после се визуализира по различен начин от Blazor и Avalonia.

Използването не остава само в ViewModel слоя. [listing 5.3](#) показва и кратък реален пример от Blazor компонента за визуализация на списъци. Компонентът е generic, работи с ObjectListViewModel<TItem>, взима видимите колони от ViewModel-a и връща избрания ред като пълен обект, а не само като стойности от таблицата.

```

1  @typeparam TItem
2
3  <table class="table object-list-table">
4      <thead>
5          <tr>
6              @foreach (var column in ViewModel.VisibleColumns)
7              {
8                  <th>@column.Header</th>
9              }
10             @if (RowActions is not null)
11             {
12                 <th>Actions</th>
13             }
14         </tr>
15     </thead>
16     <tbody>
17         @foreach (var row in ViewModel.Rows)
18         {
19             <tr class="@GetRowCssClass(row)" @onclick="() => SelectRowAsync(row)"
20             >
21                 @foreach (var cell in row.Cells)
22                 {
23                     <td>@cell.Value</td>

```

```

24         @if (RowActions is not null)
25         {
26             <td @onclick:stopPropagation="true">
27                 @RowActions(row.Item)
28             </td>
29         }
30     </tr>
31 }
32 </tbody>
33 </table>
34
35 @code {
36     [Parameter, EditorRequired]
37     public ObjectListViewModel<TItem> ViewModel { get; set; } = null!;
38
39     [Parameter]
40     public RenderFragment<TItem>? RowActions { get; set; }
41
42     [Parameter]
43     public EventCallback<ObjectListRowViewModel<TItem>> OnRowSelected { get;
44         set; }
45
46     private string GetRowCssClass(ObjectListRowViewModel<TItem> row)
47     {
48         return EqualityComparer<TItem>.Default.Equals(ViewModel.SelectedItem
49             , row.Item)
50             ? "is-selected"
51             : string.Empty;
52     }
53
54     private async Task SelectRowAsync(ObjectListRowViewModel<TItem> row)
55     {
56         ViewModel.SelectRow(row);
57
58         if (OnRowSelected.HasDelegate)
59         {
60             await OnRowSelected.InvokeAsync(row);
61         }
62     }
63 }

```

Код 5.3: Откъс от ObjectListTable.razor като generic reusable list компонент

Следващият кратък откъс показва как задачите за анализ се отделят от потребителския интерфейс. QueuedAnalysisBackgroundService периодично търси чакащ анализ, създава score за инфраструктурните услуги и делегира реалната обработка към

IAalysisRunProcessor. Така UI слойът създава задача, но не носи тежестта на самата обработка.

```
1 public sealed class QueuedAnalysisBackgroundService(
2     IServiceScopeFactory serviceScopeFactory,
3     ILogger<QueuedAnalysisBackgroundService> logger) : BackgroundService
4 {
5     protected override async Task ExecuteAsync(CancellationToken
6     stoppingToken)
7     {
8         using var timer = new PeriodicTimer(TimeSpan.FromSeconds(3));
9
10        while (!stoppingToken.IsCancellationRequested)
11        {
12            await using var scope = serviceScopeFactory.CreateAsyncScope();
13            var processor = scope.ServiceProvider.GetRequiredService<
14            IAalysisRunProcessor>();
15            var processed = await processor.ProcessNextQueuedAsync(
16            stoppingToken);
17
18            if (processed)
19            {
20                continue;
21            }
22
23            try
24            {
25                await timer.WaitForNextTickAsync(stoppingToken);
26            }
27            catch (OperationCanceledException)
28            {
29                logger.LogInformation("Queued analysis background service is
30                stopping.");
31                break;
32            }
33        }
34    }
35 }
```

Код 5.4: Откъс от QueuedAnalysisBackgroundService.cs за обработка на чакащи анализи

Последният ключов фрагмент е свързан с реалната последователност на инференцията. По-долу е показан съкратен откъс от модула, който интегрира YoloDotNet, извличането на кадри и генерирането на анотирано видео.

```
1 public async Task<AnalysisInferenceResult> RunAsync(
2     AnalysisInferenceRequest request,
3     CancellationToken cancellationToken = default)
4 {
```

```

5      cancellationToken.ThrowIfCancellationRequested();
6
7      var yoloOptions = options.Value.YoloDotNet;
8      var modelPath = GetRequiredPath(yoloOptions.ModelPath, "
ComputerVision:YoloDotNet:ModelPath");
9
10     if (string.IsNullOrEmpty(request.SourceVideoRelativePath) ||
11         !mediaStorageService.TryGetPhysicalPath(request.
SourceVideoRelativePath, out var sourceVideoPath) ||
12         !File.Exists(sourceVideoPath))
13     {
14         throw new FileNotFoundException(
15             "The uploaded source video could not be found for this
analysis run.",
16             request.SourceVideoRelativePath);
17     }
18
19     var workspace = videoProcessingService.CreateWorkspace(request.
AnalysisRunId);
20
21     try
22     {
23         var extractedFrames = await videoProcessingService.
ExtractFramesAsync(
24             sourceVideoPath,
25             workspace,
26             yoloOptions.FrameSampleIntervalSeconds,
27             yoloOptions.MaxFramesPerRun,
28             cancellationToken);
29         var allDetections = new List<AnalysisInferenceDetection>();
30
31         foreach (var frame in extractedFrames)
32         {
33             cancellationToken.ThrowIfCancellationRequested();
34
35             using var image = SKBitmap.Decode(frame.PhysicalPath)
36                 ?? throw new InvalidOperationException($"Unable to
decode extracted frame at '{frame.PhysicalPath}'");
37
38             var detections = GetOrCreateYolo(modelPath)
39                 .RunObjectDetection(
40                     image,
41                     confidence: yoloOptions.ConfidenceThreshold,
42                     iou: yoloOptions.IouThreshold)
43                 .Cast<object>()

```

```

44         .Select(detection => MapDetection(detection, frame.
FrameNumber))
45         .ToArray();
46
47         DrawDetections(image, detections);
48         await videoProcessingService.SaveAnnotatedFrameAsync(
49             image,
50             workspace,
51             frame.SequenceNumber,
52             cancellationToken);
53
54         allDetections.AddRange(detections);
55     }
56
57     var preparedOutput = mediaStorageService.
PrepareProcessedOutputPath(
58         request.OriginalFileName,
59         request.AnalysisRunId);
60
61     await videoProcessingService.RenderAnnotatedVideoAsync(
62         workspace,
63         preparedOutput.PhysicalPath,
64         yoloOptions.FrameSampleIntervalSeconds,
65         cancellationToken);
66
67     return new AnalysisInferenceResult(
68         CompletedAtUtc: DateTime.UtcNow,
69         Detections: allDetections,
70         OutputRelativePath: preparedOutput.RelativePath);
71 }
72 finally
73 {
74     videoProcessingService.CleanupWorkspace(workspace);
75 }
76 }

```

Код 5.5: Откъс от YoloDotNetAnalysisInferenceEngine.cs с основния inference pipeline

[listing 5.5](#) показва, че анализът не се свежда до запис на фиктивни стойности в базата. Първо се валидира изходното видео и се създава временна работна директория. След това кадрите се извличат чрез VideoProcessingService, върху всеки кадър се извиква RunObjectDetection, резултатите се преобразуват в удобна за домейна структура, рисуват се ограничителни рамки и накрая се рендерира изходно аотирано видео. Така аналитичната стойност на проекта е свързана с изпълним цялостен

процес, а не само с визуализация на предварително подготвени данни.

При Avalonia клиента визуализацията на медийното съдържание е реализирана чрез предварителни кадри и отваряне на оригиналното или обработеното видео със системния видео плейър. Така настолното приложение използва съществуващата `ffmpeg` последователност, без да въвежда тежка вградена зависимост за възпроизвеждане на видео.

5.6 Трудности и преодоленни проблеми

В процеса на реализация възникнаха няколко трудности, които оказаха влияние върху стабилността и завършеността на приложението. Сред тях бяха синхронизацията между Web и Avalonia при различни доставчици и различно файлово хранилище, логиката по стартиране на настолния клиент и фоновите услуги, проблеми с темата и контраста в Avalonia под Linux, поведението на локалния HTTPS сертификат за разработка в брауъра и необходимостта ясно да се разграничат статусът на видеото и статусът на анализа в интерфейса.

Един от по-полезните практически изводи беше, че архитектурно правилният код сам по себе си не е достатъчен за завършено приложение. Необходимо е системата да бъде не само правилно структурирана, но и стабилна, разбираема и без подвеждащи състояния на интерфейса. Точно поради тази причина в по-късните етапи беше обърнато внимание и на удобството за работа с Avalonia клиента, така че той да бъде реално използваем интерфейс.

Глава 6 Примери за използване

6.1 Пример за завършен анализ

Най-естественият начин за използване на системата започва с качване на локално видео. Потребителят избира файл от файловата система, приложението го записва във файловото хранилище и създава съответен запис в таблицата `Videos`. От този момент нататък видеото става част от общата библиотека и може да бъде използвано като източник за нов анализ. Ако двата клиента работят върху една и съща конфигурация, записът става видим и в веб, и в настолния интерфейс.

След като съществува качено видео, може да се създаде нов `AnalysisRun`. От гледна точка на потребителя това е кратко действие от интерфейса, но вътрешно то създава задача, която по-късно се поема от фоновия обработчик. Така потребителското действие се отделя от тежката аналитична обработка и системата остава отзивчива дори

при по-дълги операции. След приключване на анализа в базата се записват статусът, резултатите от детекцията и обработеното видео.

След завършване на анализа потребителят може да прегледа подробностите за конкретно изпълнение. Тук системата показва използвания модел, броя на разпознатите обекти, обобщение по етикети, оригиналното видео, анотираното изходно видео и таблица с отделните detection results. По този начин приложението демонстрира завършен сценарий от качване на видео до преглед на финален резултат.

Показател	Стойност
Изходно видео	crowd.mp4
Използван модел	yolov8n.onnx
Статус на анализа	Completed
Общо открити обекти	424
Основен клас	person – 409 разпознавания
Други класове	bird – 6, dog – 5, handbag – 2
Допълнителни класове	skis – 1, suitcase – 1

Таблица 6.1: Примерни стойности от завършен анализ на видео crowd.mp4

таблица 6.1 показва реален резултат от изпълнение на анализа. В този пример системата разпознава голям брой хора в кадрите, което съответства на съдържанието на видеото. Същите данни се визуализират както в Blazor клиента, така и в Avalonia клиента, когато двете приложения използват една и съща база данни и едно и също файлово хранилище.

```
1 Video uploaded: crowd.mp4
2 Analysis run created with model: yolov8n.onnx
3 Run status: Queued -> Processing -> Completed
4 Output video and detections are ready
5 Objects detected: 424
6 Top labels: person=409, bird=6, dog=5, handbag=2
```

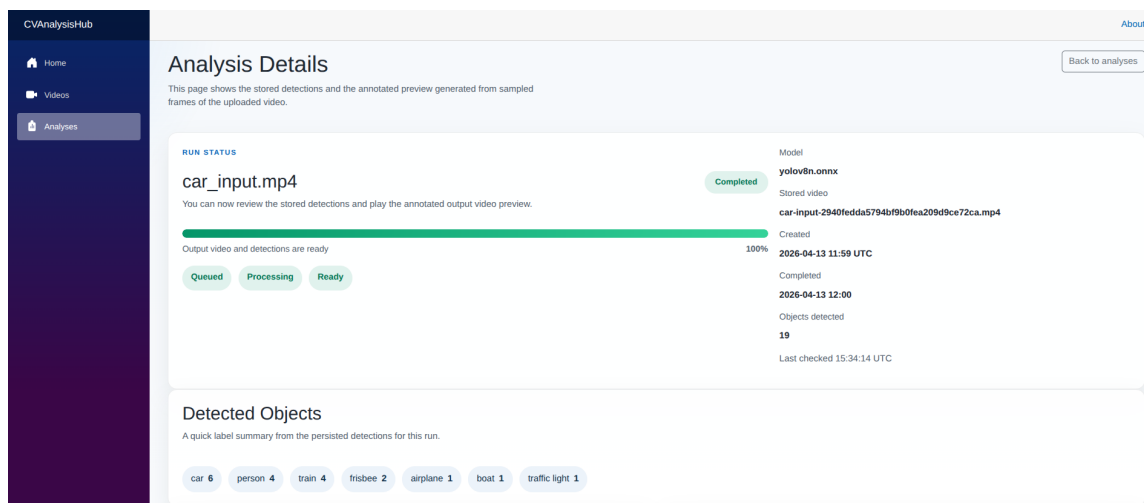
Код 6.1: Съкратен лог от изпълнението и обновяването на анализа

Логът от listing 6.1 не е пълен системен журнал, а съкратено представяне на съществени състояния в процеса на обработка. Те показват преминаването от качено видео към чакаща задача, последваща обработка, завършен анализ и записани резултати. Така примерът описва реален процес на обработка, а не визуализация на статични примерни данни.

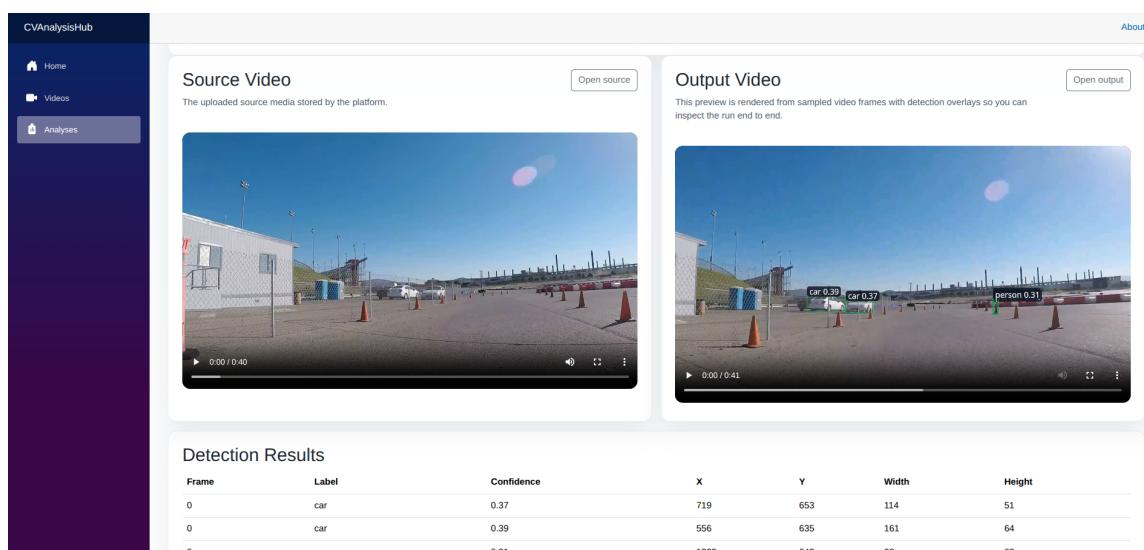
6.2 Екрани за демонстрация

Екраните в тази секция показват завършения сценарий в двата потребителски интерфейса и илюстрират, че резултатите от анализа се четат от общия приложен слой.

Blazor изгледът представя най-пълната визуализация на анализа, докато Avalonia изгледът използва същите данни и същите ViewModel-и в настолен клиент.



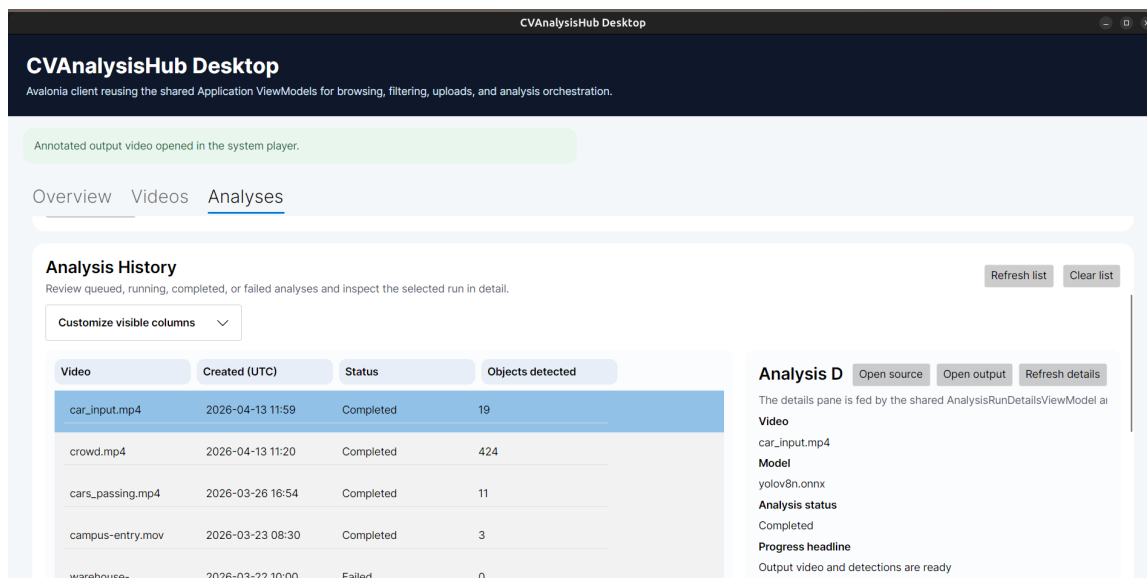
Фигура 6.1: Blazor интерфейс с подробности за завършен анализ, обобщение на разпознатите обекти и видео визуализация



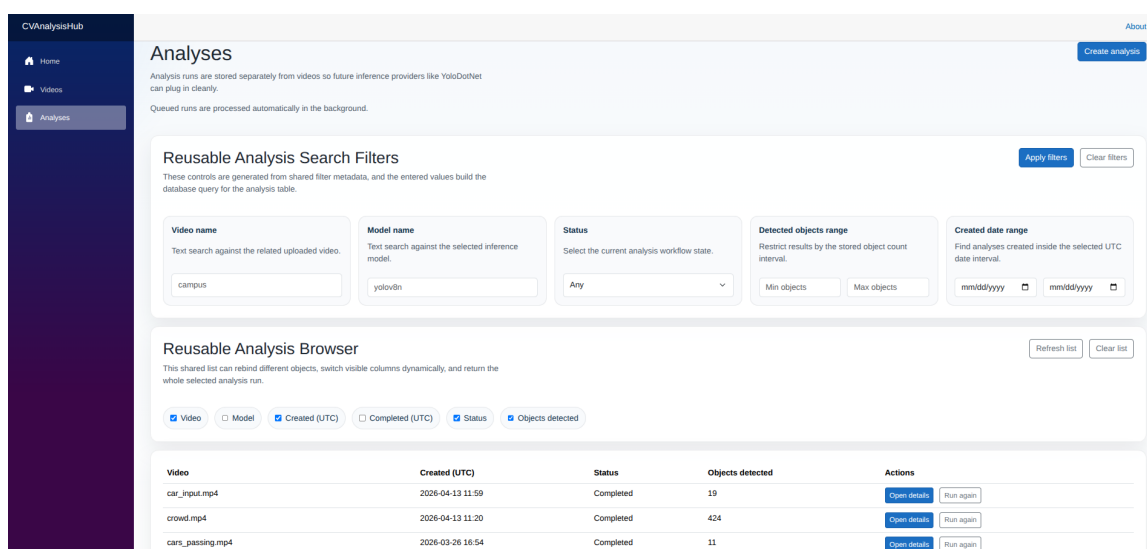
Фигура 6.2: Blazor интерфейс с таблица на отделните detection results за завършения анализ

6.3 Демонстрация на преизползване

Уеб клиентът показва преизползваемо филтриране върху два различни набора от данни. При **Videos** са реализирани филтри по име, статус, продължителност и дата на качване, а при **AnalysisRuns**, по видео, модел, статус, брой открити обекти и дата на създаване. Съществено е, че една и съща инфраструктура за филтриране работи върху различни модели без да бъде писана отново за всеки от тях.



Фигура 6.3: Avalonia интерфейс със същия завършен analysis run и синхронизирани резултати



Фигура 6.4: Демонстрация на използваемо филтриране и използваема визуализация на списък от обекти

Подобен ефект се наблюдава и при преизползваемата визуализация на списъци от обекти. Потребителят може да променя видимите колонии, да избира конкретен ред и да работи с целия съответстващ обект, независимо каква част от него е визуализирана в таблицата. Тази функционалност е приложена както за **Videos**, така и за **AnalysisRuns**, което показва, че механизмът работи върху повече от един тип данни.

Глава 7 Заключение

В крайния си вид проектът представлява работещо .NET приложение със завършен сценарий за видео анализ и ясна архитектурна организация. Системата използва MVVM, поддържа две различни View технологии, използва споделени ViewModel-и, работи с SQLite и PostgreSQL чрез общ модел и общ бизнес слой и включва преизползваеми функционалности за филтриране и визуализиране на списъци от обекти.

Най-важният архитектурен извод от разработката е, че правилното разделение по слоеве дава по-добър резултат от механично разпределение на файловете по шаблонни папки. Чрез такова разделение проектът постигна споделена приложна логика, технологично независими ViewModel-и, конфигурационно превключване между две СУБД и преизползваеми абстракции за потребителски интерфейс, без да жертва разбираемостта на реализацията.

От практическа гледна точка разработката показва, че архитектурно коректният код трябва да бъде подкрепен и от стабилно поведение в реална среда. Именно поради това в проекта беше обърнато внимание не само на модела и слоя за съхранение, но и на фоновата обработка, обработката на видео, синхронизацията между двата клиента и подобренията в Avalonia интерфейса.

Макар текущият вариант вече да е достатъчно завършен като курсов проект, системата има потенциал и за бъдещо развитие. Възможни посоки са по-богато визуализиране на резултатите, въвеждане на миграции вместо **EnsureCreated**, по-пълно изравняване на функционалността между двата клиента и допълнителни аналитични функции върху резултатите от детекцията.

Литература

- [1] CVAnalysisHub GitHub repository, <https://github.com/Apsie09/CV-Analysis-Hub>
- [2] CodeBoarding GitHub repository, <https://github.com/CodeBoarding/CodeBoarding>
- [3] Avalonia UI documentation, <https://docs.avaloniaui.net/>

- [4] Microsoft Learn, Blazor documentation, <https://learn.microsoft.com/aspnet/core/blazor/>
- [5] Microsoft Learn, Entity Framework Core documentation, <https://learn.microsoft.com/ef/core/>
- [6] SQLite Documentation, <https://www.sqlite.org/docs.html>
- [7] PostgreSQL Documentation, <https://www.postgresql.org/docs/>
- [8] PlantUML official documentation, <https://plantuml.com/>
- [9] PlantUML LaTeX support, <https://plantuml.com/latex>
- [10] VSCode Marketplace, PlantUML extension by jebbs, <https://marketplace.visualstudio.com/items?itemName=jebbs.plantuml>
- [11] GitHub repository chippyash/db-plantuml, <https://github.com/chippyash/db-plantuml>
- [12] YoloDotNet repository / package documentation, <https://github.com/NickSwardh/YoloDotNet>
- [13] Ultralytics, “Introducing Ultralytics YOLOv8”, <https://www.ultralytics.com/blog/introducing-ultralytics-yolov8>
- [14] Ultralytics YOLO model documentation and model comparison tables, <https://huggingface.co/Ultralytics/YOLOv8>
- [15] FFmpeg Documentation, <https://ffmpeg.org/documentation.html>